Unit 8 Lecturecast - Backpropagation in Artificial Neural Networks (ANNs)

Lesson 1 - Reducing the Error

Recall that in order for a neural networks to learn, weights associated with neuron connections must be updated after forward passes of data through the network. These weights are adjusted to help reconcile the differences between the actual and predicted outcomes for subsequent forward passes. Since we are talking about the difference between actual and predicted values, the error would be a useful measure here. So, each neuron will require that their respective error be sent backward through the network to them, in order to facilitate the update process, hence the name, backpropagation of error. Backpropagation, short for “backward propagation of errors”, is a mechanism used to update the weights using gradient descent. It calculates the gradient of the error function with respect to the neural network’s weights. The calculation proceeds backwards through the network.

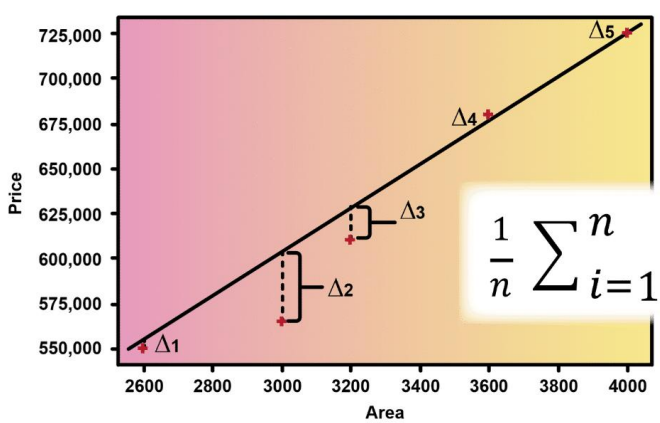
Visualise backpropagation and weight update

- But why not take a guess about weights and try and test?
- This simple 1 hidden layer network has 16 weights and with a weight of 3 decimal point precision, can have 1048 weight combinations.
- Computationally, it is costly to try and test all 1048 combinations and find the optimal 16 weights with least error.

Our main goal of the training is to reduce the **error** or the difference between **prediction** and **actual output**. Since the **actual output** is constant, that is “not changing”, the only way to reduce the error is to change the **prediction** value. The question now is, how to achieve minimal error or a prediction value close to actual output without trying all 10^48 combinations?

Lesson 2 - What is Error

The error can be defined as the difference between the actual output and the predicted output at each data point. Mean squared error or MSE defines this value with the equation. Since the value of the regression straight line can be represented by the straight line formula y = mx + b, the equation for MSE can be rewritten as shown.



$$\frac{1}{n}\sum_{i=1}^{n}(\Delta i)^2$$

$$MSE = \frac{1}{n}\sum_{i=0}^{n}(y_{i\,(actual)} - y_{predicted})^2$$

$$MSE = \frac{1}{n}\sum_{i=0}^{n}(y_i - (mx_i + b))^2$$

Lesson 3 - Gradient Descent

In order to decrease the error, both the slope m and the intercept y need to be changed. If we plot the changing slope and intercept against the MSE in a 3D plot, we will have a hyperplane like this. With both m & b being 0, the cost is high at 1000. However, as the value of m and b increases the cost start to decrease to a point where it is minimum and after that with further increase of m & b the value of MSE starts to increase. The objective is therefore to find this optimal value where cost is minimum. Gradient descent is used for this purpose.

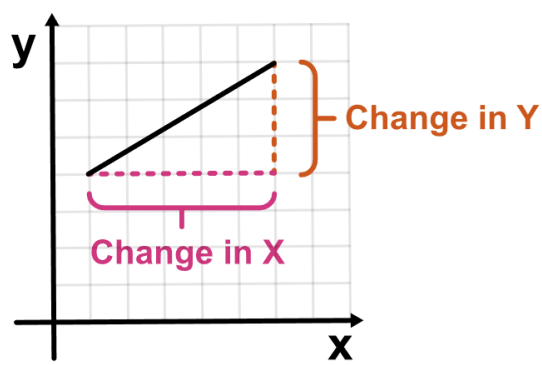
Gradient descent is an iterative optimisation algorithm used to find the minimum of a function; in our case, we want to minimize the error function. To find a local minimum of a function using a gradient descent, one takes steps proportional to the negative of the gradient of the function at the current point.

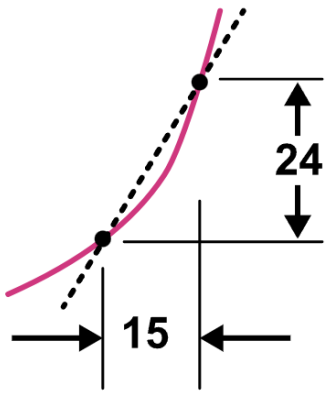
If we take one variable, in this case, the intercept, the local minima can be found by moving the variable value step wise and each time calculate the MSE. From a computational point of view, if we take a fixed size step, it would be easier but in that case, there is a chance of missing the local minima of MSE. Therefore, we need to take an approach where the step size is related to the slope change of the line. At the initial stage, with the big step size, the slope will not change that much but as the MSE starts to reach local minima, step size will increase the slope change value. In such a case, we will take small step sizes to identify the exact local minima. Slope changes, also known as **derivates**, enable us to quantify change.

Lesson 4 - Derivatives

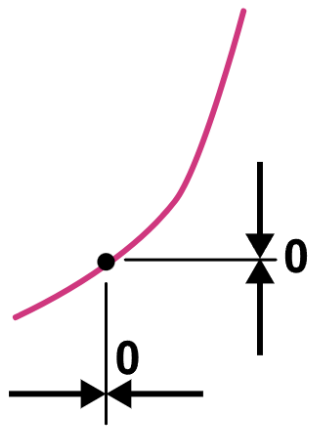
Derivatives basically enable us to measure small changes in a slope as well as provide the direction of the line. For a recap on derivatives, see the Numerical Analysis module or review this website.

$$\text{Slope} = \frac{\text{Change in Y}}{\text{Change in X}}$$

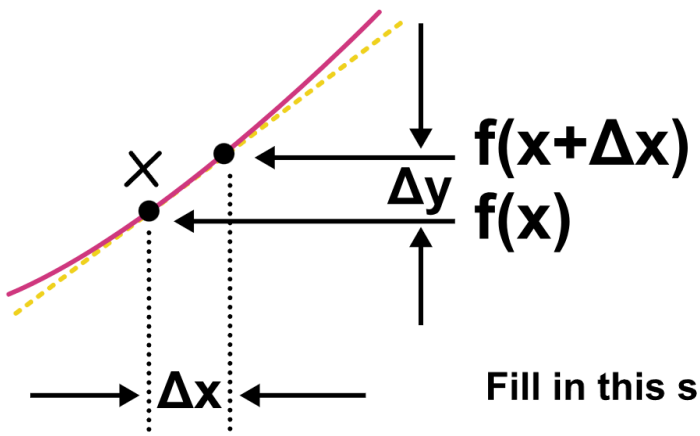




Average slope = $\frac{24}{15}$



Slope = $\frac{0}{0}$ = ???



Fill in this slope formula: $\frac{\Delta y}{} = \frac{f(x + \Delta x) - f(x)}{\Delta x}$

The slope formula is: $\frac{f(x+\Delta x) - f(x)}{\Delta x}$

Put in f(x+Δx) and f(x): $\frac{x^2 + 2x\,\Delta x + (\Delta x)^2 - x^2}{\Delta x}$

Simplify (x^2 and $-x^2$ cancel): $\frac{2x\,\Delta x + (\Delta x)^2}{\Delta x}$

Simplify more (divide through by Δx: $= 2x + \Delta x$

Then, as Δx heads towards 0 we get: $= 2x$

One variable derivatives n X $^{n-1}$

$$f(x) = x^3$$

$$\frac{d}{dx} x^3 = 3x^2$$

Two variable derivatives → partial derivatives of x, y

$$f(x,y) = x^3 + y^2$$

$$\partial f/_{\partial x} = 3x^2 + 0 = 3x^2$$

$$\partial f/_{\partial y} = 0 + 2y = 2y$$

Using the two variables derivatives formula, we can solve the MSE equation which gives the direction of the slope. Now, we know the direction we are going we need to know the step size or learning rate.

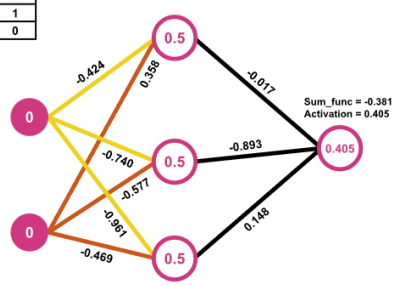
Now that we know the direction from the derivatives of m & b, we need to determine how fast or slow we will go. In other words how big our steps will be. Learning rate parameters provide us the capability to change the step size. The idea is to reduce the cost at each step through manipulating the learning rate.

For adjusting the weights within the network, we use this gradient approach where we use the derivatives of the network's weights with respect to the output error. This is called **the delta rule** and we use this delta rule in the general backpropagation algorithm.

Lesson 5 - Delta Rule

Output Layer - Delta

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



$\text{Delta}_{\text{output}}$ = error * $\text{sigmoid}_{\text{derivative}}$

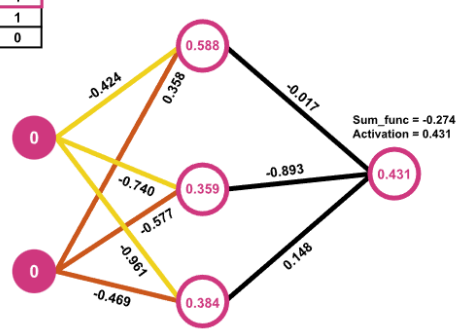
Error = 0 – 0.405 = -0.405
Derivative activation (sigmoid) = 0.241
Delta (output) = -0.405 * 0.241 = -0.097

```
import numpy as np
def sigmoid(x):
    return 1/(1 + np.exp(-x))

def der_sigmoid(x):
return sigmoid(x) * (1- sigmoid(x))

print (der_sigmoid (0.405))
```

0.2400223137656898

Using the delta rule first calculate the delta output from outer layer towards the hidden layer.

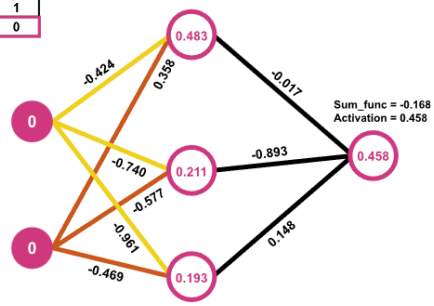
x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



$\text{Delta}_{\text{output}}$ = error * $\text{sigmoid}_{\text{derivative}}$

Error = 1 – 0.431 = 0.569
Derivative activation (sigmoid) = 0.245
Delta (output) = 0.569 * 0.245 = 0.139

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0

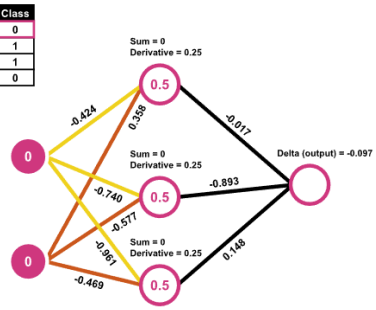


$\text{Delta}_{\text{output}}$ = error * $\text{sigmoid}_{\text{derivative}}$

Error = 0 – 0.458 = -0.458
Derivative activation (sigmoid) = 0.248
Delta (output) = -0.458 * 0.248 = -0.113

Hidden Layer - Delta

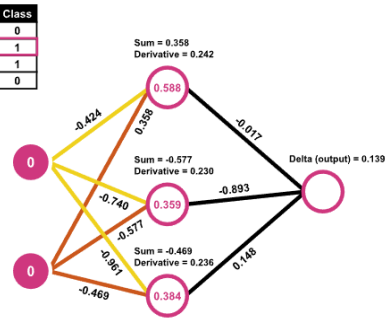
x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



$\text{Delta}_{\text{hidden}}$ = $\text{sigmoid}_{\text{derivative}}$ * Weight * $\text{delta}_{\text{output}}$

0.25 * -0.017 * -0.097 = 0.000
0.25 * -0.893 * -0.097 = 0.021
0.25 * 0.148 * -0.097 = -0.003

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



$\text{Delta}_{\text{hidden}}$ = $\text{sigmoid}_{\text{derivative}}$ * Weight * $\text{delta}_{\text{output}}$

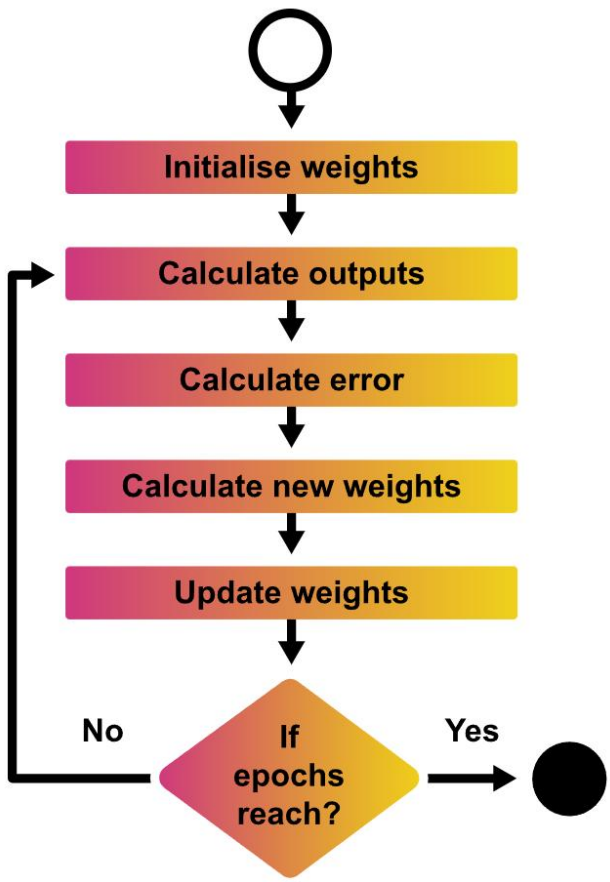
0.242 * -0.017 * 0.139 = -0.000
0.230 * -0.893 * 0.139 = -0.028
0.236 * 0.148 * 0.139 = 0.004

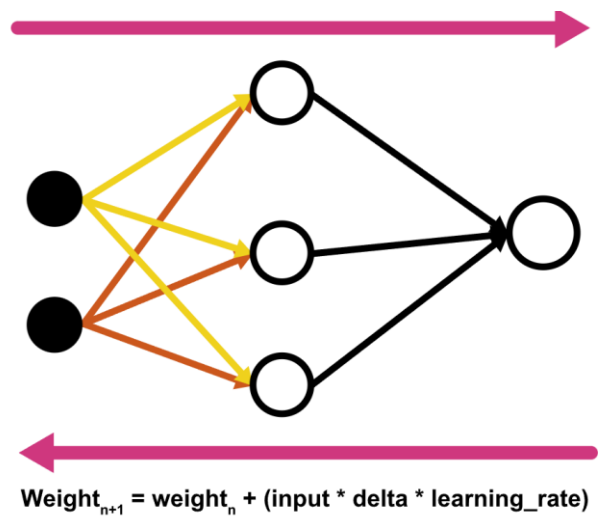
We repeat the same process for the other data input to compute the Hidden Layer - Delta.

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0

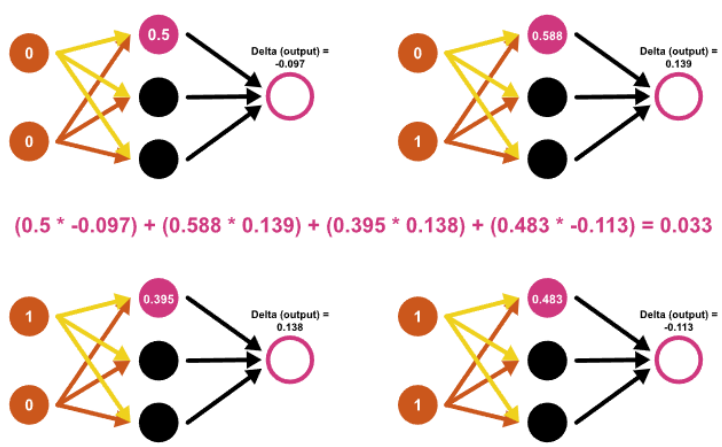
Lesson 6 - Weight Update

Now that we have calculated the delta, we start to calculate the new weights and updates through backpropagation.



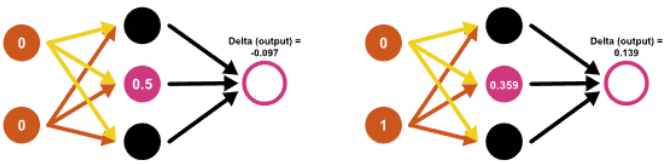


Weight Update – Output Layer To Hidden Layer



(0.5 * -0.097) + (0.588 * 0.139) + (0.395 * 0.138) + (0.483 * -0.113) = 0.033

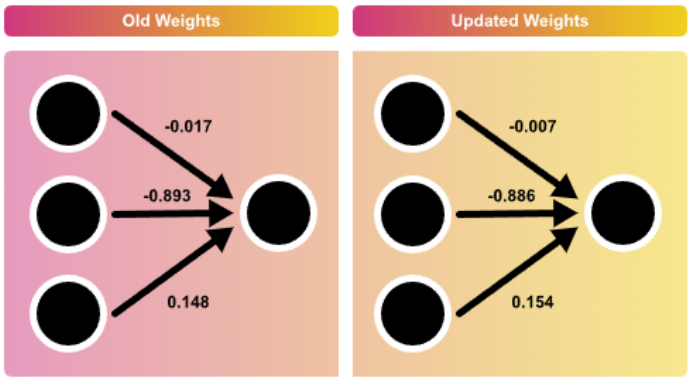
$Weight_{n+1} = weight_n + (input * delta * learning_rate)$



(0.5 * -0.097) + (0.359 * 0.139) + (0.323 * 0.138) + (0.211 * -0.113) = 0.022

$Weight_{n+1} = weight_n + (input * delta * learning_rate)$

(0.5 * -0.097) + (0.384 * 0.139) + (0.276 * 0.138) + (0.193 * -0.113) = 0.021



Our selected Learning rate is = 0.3

Result from computing: input * delta

0.033

0.022

0.021

$Weight_{n+1} = weight_n + (input * delta * learning_rate)$

-0.017 + 0.033 * 0.3 = -0.007

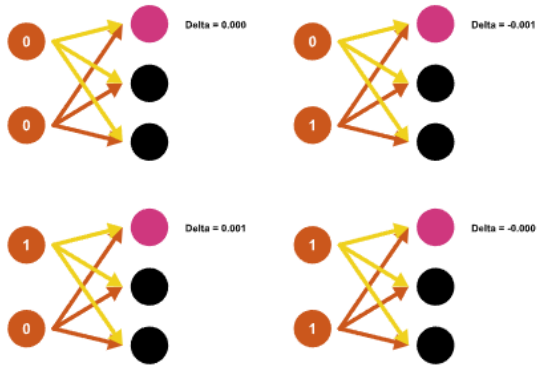
-0.893 + 0.022 * 0.3 = -0.886

0.148 + 0.021 * 0.3 = 0.154

Weight Update – Hidden Layer to Input Layer

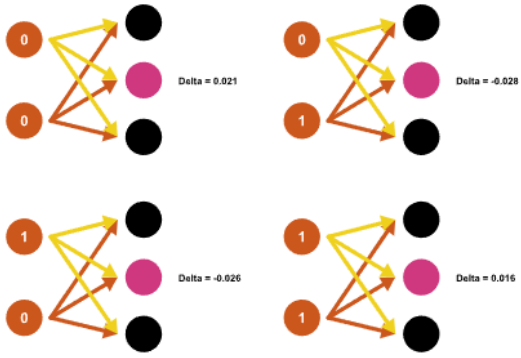
$Weight_{n+1} = weight_n + (input * delta * learning_rate)$

(0 * 0.000) + (0 * -0.001) + (1 * 0.001) + (1 * -0.000) = -0.000
(0 * 0.000) + (1 * -0.001) + (0 * 0.001) + (1 * -0.000) = -0.000



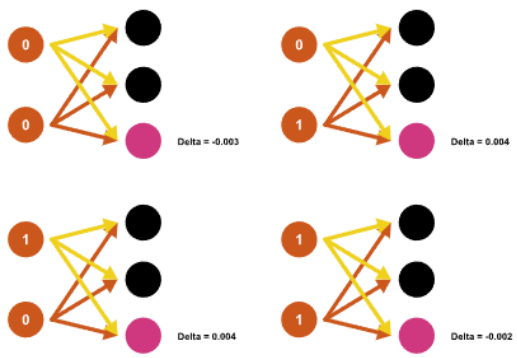
$Weight_{n+1} = weight_n + (input * delta * learning_rate)$

(0 * 0.021) +(0 * -0.028) + (1 * -0.026) + (1 * 0.016) = -0.009
(0 * 0.021) + (1 * -0.028) + (0 * -0.026) + (1 * 0.016) = -0.012

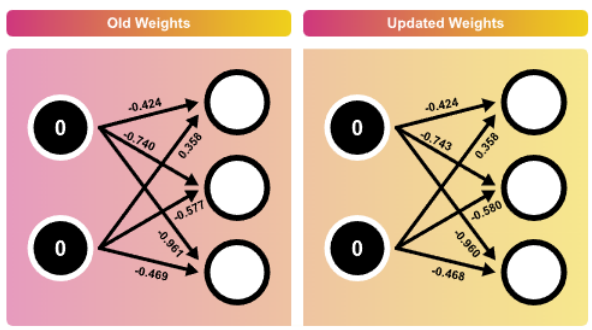


$$\text{Weight}_{n+1} = \text{weight}_n + (\text{input} * \text{delta} * \text{learning_rate})$$

$$(0 * -0.003) + (0 * 0.004) + (1 * 0.004) + (1 * -0.002) = 0.002$$
$$(0 * -0.003) + (1 * 0.004) + (0 * 0.004) + (1 * -0.002) = 0.002$$



Weight Update – Hidden Layer to Input Layer



Our selected Learning rate is = 0.3

Result from computing: input * delta

-0.000 -0.009 0.002

-0.000 -0.012 0.002

$\text{Weight}_{n+1} = \text{weight}_n + (\text{input} * \text{delta} * \text{learning_rate})$

-0.424 + (-0.000 * 0.3) = -0.424

0.358 + (-0.000 * 0.3) = 0.358

-0.740 + (-0.009 * 0.3) = -0.743

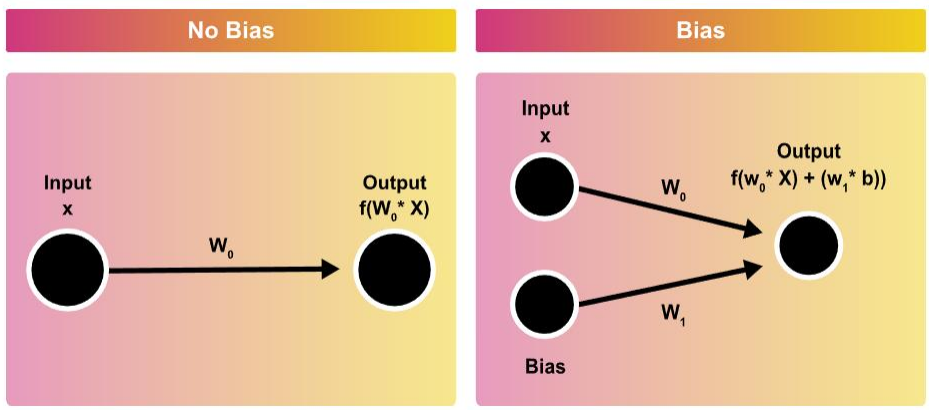
-0.577 + (-0.012) * 0.3 = -0.580

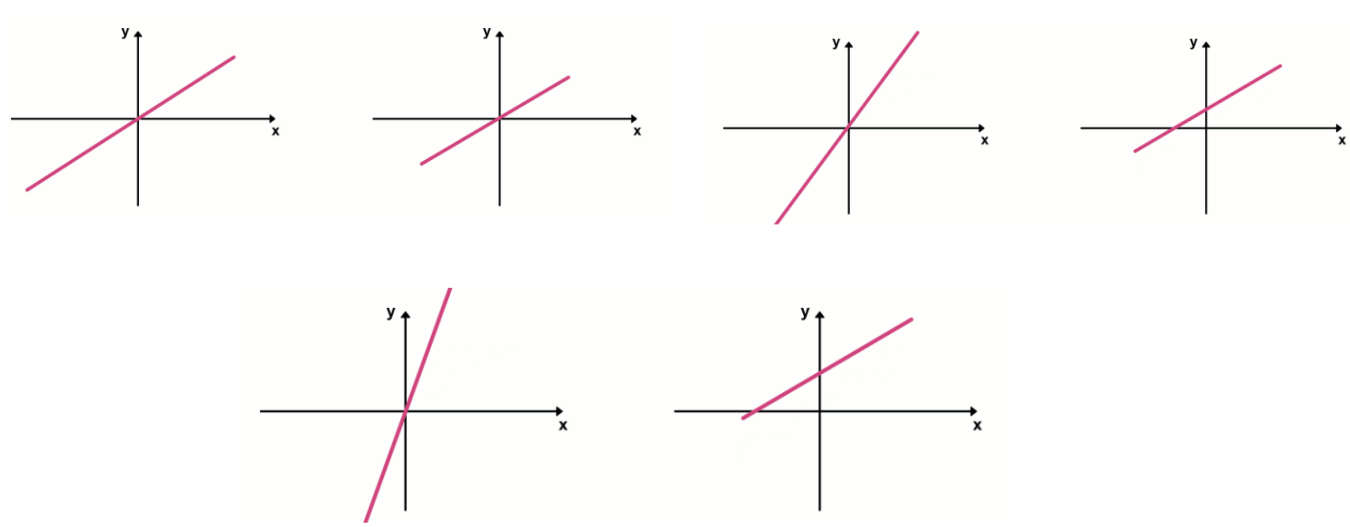
-0.961 + 0.002 * 0.3 = -0.960

-0.469 + 0.002 * 0.3 = -0.468

Lesson 7 – Bias

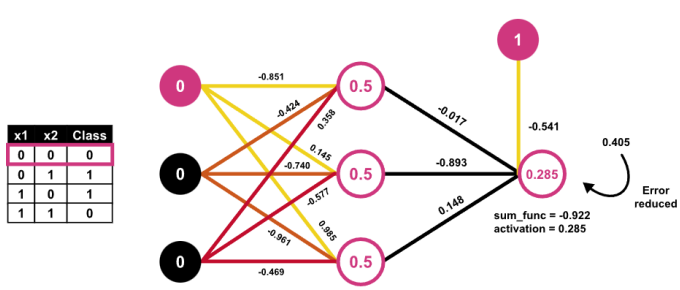
Bias in Neural Networks can be thought of as analogous to the role of a constant in a linear function, whereby the line is effectively transposed by the constant value (no gradient change). Without these spare bias weights, our model has quite limited “movement” while searching through solution space.





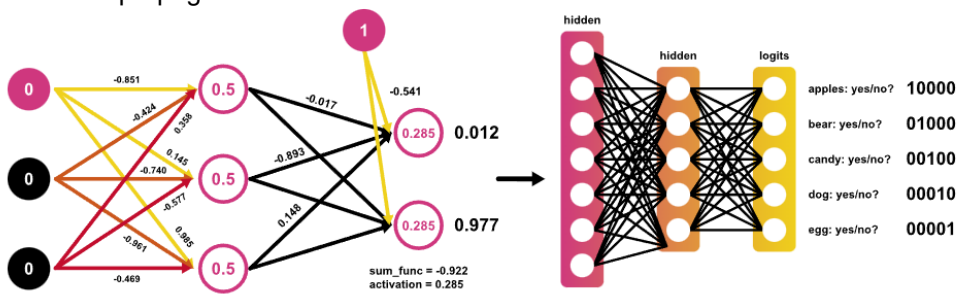
Output = sum(inputs * weights) + bias ***Source: Gebel, 2020***

With updated weights and a constant bias, we do a forward pass and reduce the error.



Lesson 7 – Multiple Outputs

So far, we have covered a NN with one hidden layer. In reality, NNs have multiple hidden layers through which backpropagation occurs.



References

Gebel, L. (2020) Why We Need Bias in Neural Networks. Available from: **https://towardsdatascience.com/why-we-need-bias-in-neural-networks-db8f7e07cb98** [Accessed on 17 May 2022]

Masri, A. (2019) How Does Back-Propagation in Artificial Neural Networks Work? Available from: https://towardsdatascience.com/how-does-back-propagation-in-artificial-neural-networks-work-c7cad873ea7 [Accessed on 17 May 2022]

Mazur, M. (2015) A Step by Step Backpropagation Example. Available from: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/ [Accessed on 17 May 2022]

Hutson, M. (2021) Robo-writers: the rise and risks of language-generating AI. Nature 591, 22-25. Available from: https://doi.org/10.1038/d41586-021-00530-0 [Accessed on 17 May 2022]

Mach, P. (2021) 10 Business Applications of Neural Network. Available from: https://www.ideamotive.co/blog/business-applications-of-neural-network [Accessed on 17 May 2022]

CDEI (2019) Snapshot Paper - AI and Personal Insurance. Available from: https://www.gov.uk/government/publications/cdei-publishes-its-first-series-of-three-snapshot-papers-ethical-issues-in-ai/snapshot-paper-ai-and-personal-insurance [Accessed on 17 May 2022]